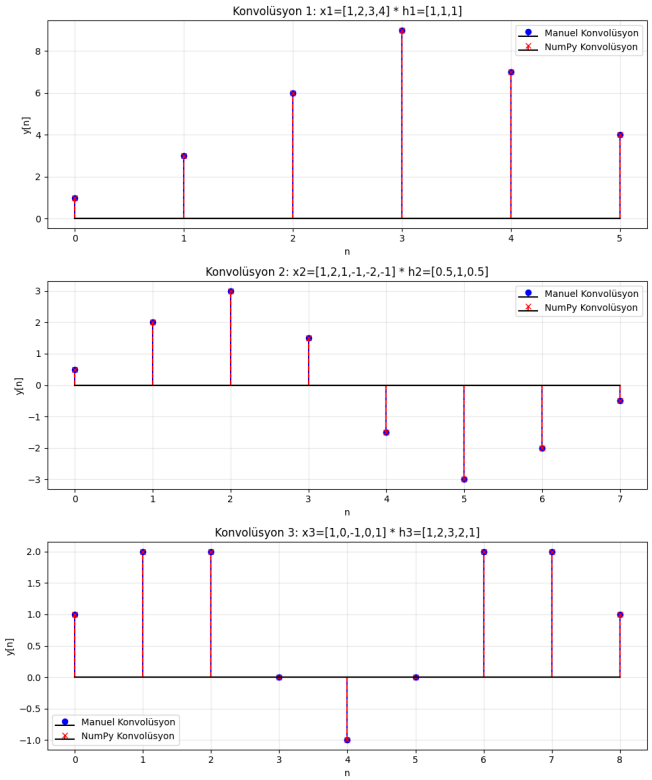
Biyomedikal İşaret ve Görüntü İşleme Ödeb 1: Konvolüsyon Ödevi

Muhammed Kayra Bulut - 25501805

Ekim 2025

1 Konvolüsyon İşlemi

Bu çalışmada, farklı sinyal ve filtre kombinasyonları ile konvolüsyon işlemleri gerçekleştirilmiştir. Aşağıdaki şekilde üç farklı konvolüsyon örneği gösterilmektedir.



Şekil 1: Konvolüsyon İşlemlerinin Karşılaştırılması

1.1 Konvolüsyon Sonuçlarının Analizi

Şekil 1'de görüldüğü üzere:

Konvolüsyon 1: $x1 = [1, 2, 3, 4] * h1 = [1, 1, 1]$ işleminde, manuel ve NumPy konvolüsyon sonuçları tam olarak örtüşmektedir. Bu işlem, giriş sinyalinin hareketli ortalama filtresi ile işlenmesini temsil eder ve çıkış sinyali düzgün bir artış göstermektedir.

Konvolüsyon 2: $x2 = [1, 2, 1, -1, -2, -1] * h2 = [0.5, 1, 0.5]$ işleminde, pozitif ve negatif değerler içeren bir sinyal işlenmiştir. Çıkış sinyalinde hem pozitif hem negatif tepe noktaları gözlemlenmektedir.

Konvolüsyon 3: $x3 = [1, 0, -1, 0, 1] * h3 = [1, 2, 3, 2, 1]$ işleminde, simetrik bir filtre kullanılmıştır. Sonuç sinyalinde belirgin tepe noktaları ve sıfır geçişleri görülmektedir.

Tüm üç durumda da manuel hesaplama ile NumPy kütüphanesi sonuçları birebir eşleşmekte, bu da konvolüsyon işleminin doğru bir şekilde gerçekleştirildiğini göstermektedir.

2 Kod

Kod aşağıda verilmiştir:

Listing 1: Konvolüsyon İşlemi Python Kodu

```
import numpy as np
import matplotlib.pyplot as plt
from typing import List, Tuple

def konvolusyon(x: np.ndarray, h: np.ndarray) -> np.ndarray:
    """
    İki dizinin (1D) konvolüsyonunu manuel olarak hesaplar.
    """
    len_x = len(x)
    len_h = len(h)
    len_y = len_x + len_h - 1

    y = np.zeros(len_y, dtype=float)

    for n in range(len_y):
        for k in range(len_h):
            if 0 <= n - k < len_x:
                y[n] += x[n - k] * h[k]
    return y

def ciz_konvolusyon(ax: plt.Axes, x: np.ndarray, h: np.ndarray,
                    baslik: str) -> None:
    """
    Verilen Axes üzerinde:
      - manuel konvolüsyon (konvolusyon)
      - NumPy built-in konvolüsyon (np.convolve)
```

```
    hesaplar ve ikisini birlikte çizer.
    """
    # Hesaplamalar
    y_manuel = konvolusyon(x, h)
    y_builtin = np.convolve(x, h)
    n = np.arange(len(y_manuel))

    # Çizimler
    ax.stem(n, y_manuel, linefmt='b-', markerfmt='bo',
           basefmt='k-', label='Manuel Konvolüsyon')
    ax.stem(n, y_builtin, linefmt='r--', markerfmt='rx',
           basefmt='k-', label='NumPy Konvolüsyon')
    ax.set_xlabel('n')
    ax.set_ylabel('y[n]')
    ax.set_title(baslik)
    ax.legend()
    ax.grid(True, alpha=0.3)

def main():
    # Dizi çiftleri ve başlıklar
    pairs: List[Tuple[np.ndarray, np.ndarray, str]] = [
        (np.array([1, 2, 3, 4], dtype=float),
         np.array([1, 1, 1], dtype=float),
         'Konvolüsyon 1: x1=[1,2,3,4] * h1=[1,1,1]'),

        (np.array([1, 2, 1, -1, -2, -1], dtype=float),
         np.array([0.5, 1, 0.5], dtype=float),
         'Konvolüsyon 2: x2=[1,2,1,-1,-2,-1] * h2=[0.5,1,0.5]'),

        (np.array([1, 0, -1, 0, 1], dtype=float),
         np.array([1, 2, 3, 2, 1], dtype=float),
         'Konvolüsyon 3: x3=[1,0,-1,0,1] * h3=[1,2,3,2,1]'),
    ]

    # Şekil/ekseni oluşturma
    fig, axes = plt.subplots(len(pairs), 1,
                            figsize=(10, 4 * len(pairs)))
    if len(pairs) == 1:
        axes = [axes]

    for ax, (x, h, baslik) in zip(axes, pairs):
        ciz_konvolusyon(ax, x, h, baslik)

    plt.tight_layout()
    plt.savefig("konvolusyon_sonuclari.png")

if __name__ == "__main__":
    main()
```